

SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09

COURSE NAME: Programming in Java

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10

Total Marks: 100

ANNEXURE A Coding Challenge

Code of Conduct:

*I **B.ANJALI(192371062)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)

2. Employee Payroll System using Inheritance

Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

3. Vehicle Rental System using Runtime Polymorphism

Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

4. Bank Account System using Data Hiding

Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

7. Hospital Management System using Interfaces

Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor.

Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

8. Online Shopping Order System using Packages

Create two packages:

- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

STUDENT NAME:B.Anjali

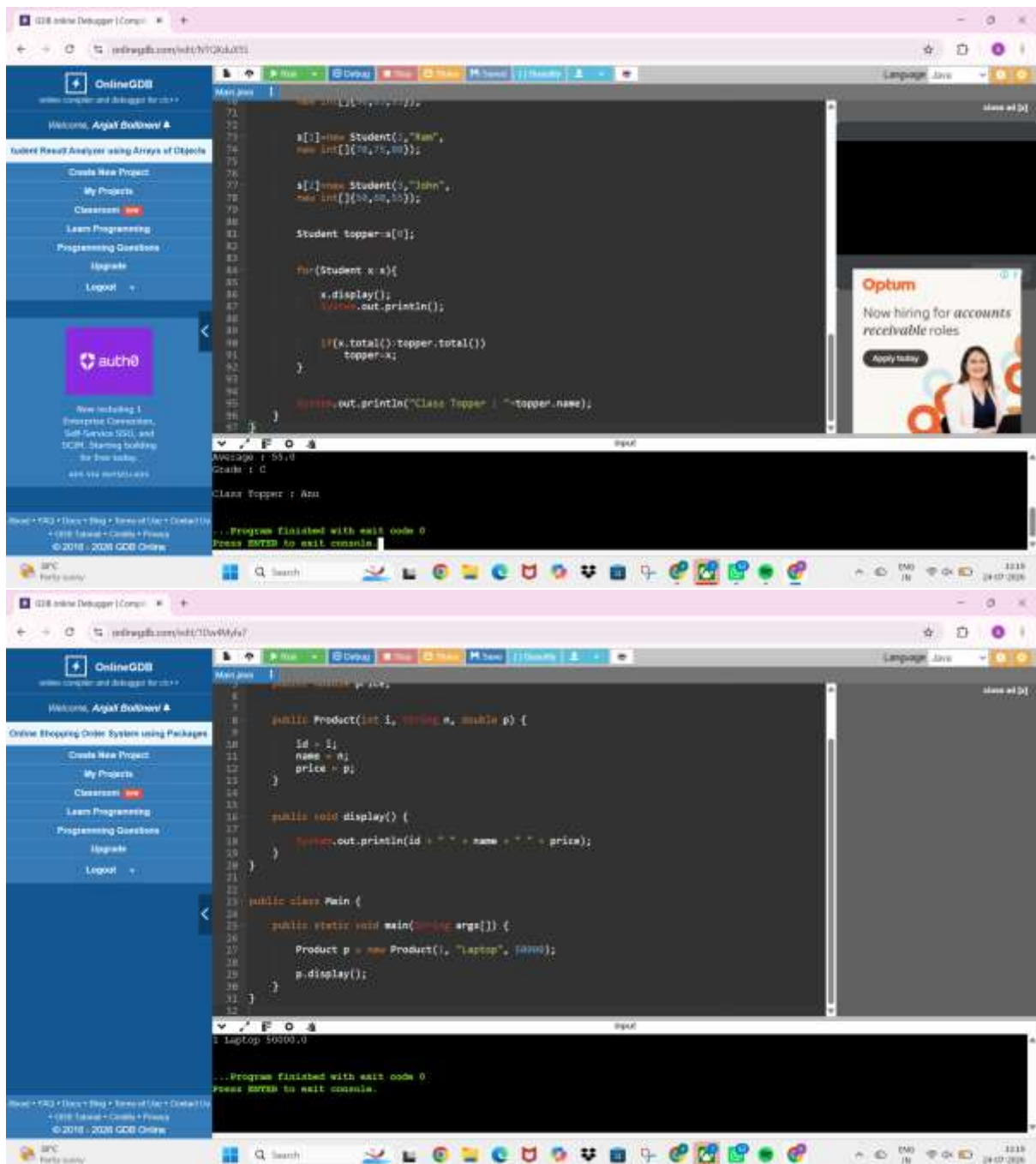
REGISTER NUMBER:192371062

COURSE CODE: CSA0903

COURSE NAME: Programming in java

GUIDED BY: Magesh Kumar

DATE:24-07-2026

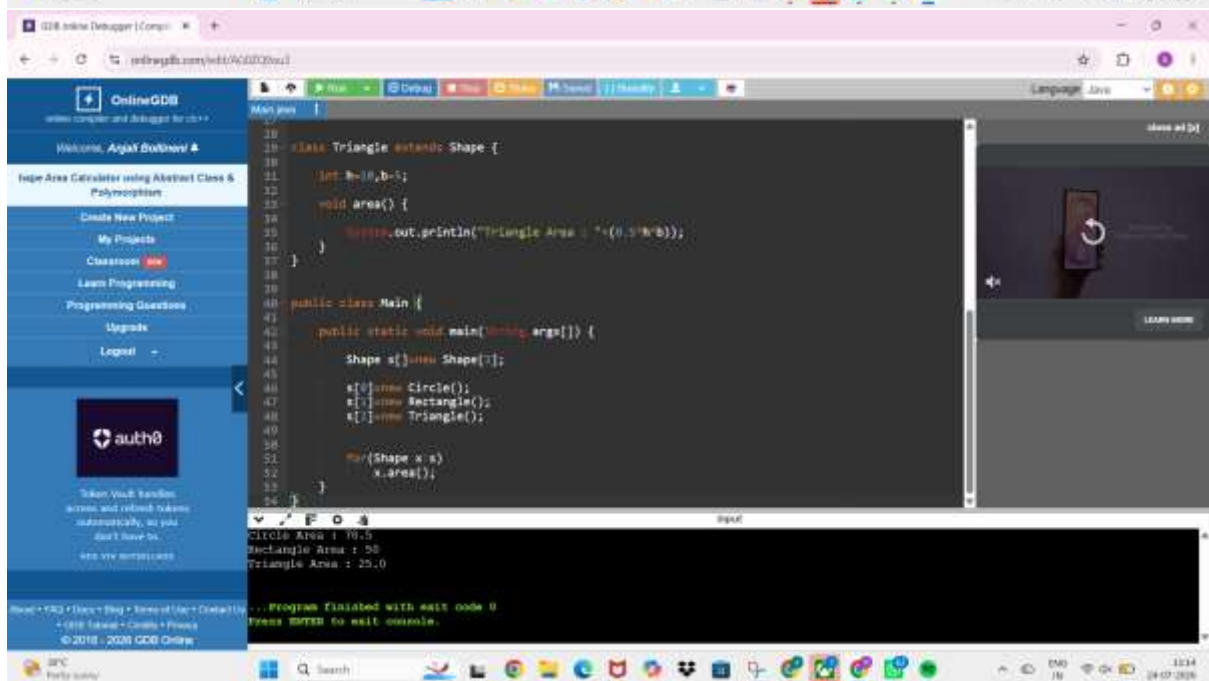


The first screenshot shows a Java program in the OnlineGDB IDE. The code defines a `Student` class with attributes `name`, `marks`, and `total`. It includes methods `display` and `total`. The `main` method creates two `Student` objects, `topper` and `runner`, and compares their total marks. The output shows the details of the topper.

```
1 // Student class
2 class Student {
3     String name;
4     int marks;
5     int total;
6
7     Student(String n, int m) {
8         name = n;
9         marks = m;
10    }
11
12    void display() {
13        System.out.println("Name: " + name + " Marks: " + marks);
14    }
15
16    int total() {
17        total = marks;
18        return total;
19    }
20 }
21
22 // Main class
23 class Main {
24     public static void main(String[] args) {
25         Student topper = new Student("Ran", 75);
26         Student runner = new Student("John", 60);
27
28         topper.display();
29         runner.display();
30
31         if (topper.total() > runner.total()) {
32             System.out.println("Class Topper : " + topper.name);
33         }
34     }
35 }
```

The second screenshot shows a Java program in the OnlineGDB IDE. The code defines a `Product` class with attributes `id`, `name`, and `price`. It includes a `display` method. The `main` method creates a `Product` object and calls the `display` method. The output shows the details of the product.

```
1 // Product class
2 class Product {
3     int id;
4     String name;
5     double price;
6
7     Product(int i, String n, double p) {
8         id = i;
9         name = n;
10        price = p;
11    }
12
13    void display() {
14        System.out.println(id + " " + name + " " + price);
15    }
16 }
17
18 // Main class
19 public class Main {
20     public static void main(String[] args) {
21         Product p = new Product(1, "Laptop", 50000);
22         p.display();
23     }
24 }
```



OnlineGDB
online compiler and debugger for c++

Welcome, Arjall Bollinani

Vehicle Rental System using Runtime Polymorphism

Create New Project
My Projects
Classroom **new**
Learn Programming
Programming Questions
Upgrade
Logout

```
16 class Bus extends Vehicle{
17     Bus(){
18         super("Bus");
19     }
20     int calculateRent(int days){
21         return days*1000;
22     }
23 }
24 public class Main{
25     public static void main(String args[]){
26         Vehicle v[]=new Vehicle[3];
27         v[0]=new Car();
28         v[1]=new Bike();
29         v[2]=new Bus();
30         for(Vehicle x:v)
31         {
32             System.out.println(x.name+" Rent = "+x.calculateRent());
33         }
34     }
35 }
```

Car Rent = 5000
Bike Rent = 2500
Bus Rent = 10000

... Program finished with exit code 0
Press ENTER to exit console

100°C
Party today

Adobe Creative Cloud Pro
Build career-ready skills with pro apps.
Now 12% off for students
Learn more

Smooth-sailing through 20+ apps.
Pay via UPI
Buy now

Adobe Creative Cloud Pro

OnlineGDB
online compiler and debugger for c++

Welcome, Arjall Bollinani

Library Book Management using Classes & Encapsulation

Create New Project
My Projects
Classroom **new**
Learn Programming
Programming Questions
Upgrade
Logout

```
1 class Book {
2     private int bookId;
3     private String title;
4     private String author;
5     private double price;
6     private boolean issued;
7
8     Book(int id, String t, String a, double p) {
9         bookId = id;
10        title = t;
11        author = a;
12        price = p;
13        issued = false;
14    }
15
16    public int getBookId() {
17        return bookId;
18    }
19
20    public String getTitle() {
21        return title;
22    }
23
24    public void issueBook() {
25        if(issued) {
26            issued = true;
27            System.out.println(title + " issued successfully");
28        }
29    }
30 }
```

Java issued successfully
Java returned successfully
Search Book:
Book Found

```
Main.java
30
31 class ContractEmployee extends Employee{
32
33     ContractEmployee(String n,int i){
34         super(n,i);
35     }
36
37     double calculateSalary(){
38         return 25000;
39     }
40 }
41
42 public class Main{
43     public static void main(String args[]){
44
45         Employee e[];
46
47         e=new Employee[2];
48
49         e[0]=new PermanentEmployee("Anu",101);
50         e[1]=new ContractEmployee("Ram",102);
51
52         for(Employee x:e){
53             x.display();
54             System.out.println("Salary : "+x.calculateSalary());
55         }
56     }
57 }
```

input

Anu 101
Salary : 50000.0
Ram 102
Salary : 25000.0

OnlineGDB

Welcome, Anjali Bhatnagar

Smart Home Device Controller using Hierarchical Inheritance

Create New Project

My Projects

Courses

Learn Programming

Programming Questions

Upgrade

Logout

auth0

Get your app's bag in, call API, and connect to API services securely with Auth0.

API V1X 5075011495

Read • VS2 • Next Step • View of User Details • GDB Tutorial • Credits • Privacy • © 2018 - 2020 GDB Online

```
Main.java
90 Device d[]=new Device[3];
91
92 d[0]=new Light();
93 d[1]=new Fan();
94 d[2]=new AirConditioner();
95
96
97
98
99
100
101 for(Device x:d){
102     x.turnOn();
103     System.out.println(
104         "Power Consumption : "
105         +x.powerConsumption()+" Watts");
106
107     x.turnOff();
108     System.out.println();
109 }
110 }
```

input

Light ON
Power Consumption : 20 Watts
Light OFF

Fan ON
Power Consumption : 40 Watts
Fan OFF

Optum

Now hiring for accounts receivable roles

Apply today

12:20 24-07-2020